

Sprint 1 — Kişi 2: Auth, Profil ve Kuyruk API'si

Bu belge, Sprint 1'de Kişi 2 tarafından teslim edilen backend'i özetler: kimlik doğrulama (auth), profil/tercihler ve eşleştirme kuyruğu (queue) API'leri. Tüm kod `server/` klasörü altındadır ve mevcut istemci (Kişi 1) ile CI/WebRTC (Kişi 3) alanlarına dokunmaz.

Teknoloji

- Node.js 22 + TypeScript (strict mode)
- Fastify 5 + `@fastify/jwt` (JWT ile kimlik doğrulama)
- PostgreSQL (kalıcı veri) — `pg` sürücüsü
- Redis (`ioredis`) — ileride paylaşımlı hız sınırı/kuyruk mevcudiyeti için ayrıldı
- Zod (istek doğrulama), Vitest (test)

Çalıştırma

```
cd server
npm install
# .env dosyasında DATABASE_URL ve SESSION_JWT_SECRET tanımlı olmalı
psql "$DATABASE_URL" -f src/db/schema.sql # şemayı uygula (migration)
npm run dev                               # geliştirme
npm run build && npm start                 # üretim
```

Ortam değişkenleri (`.env.example`)

Değişken	Sahip	Açıklama
<code>DATABASE_URL</code>	API (Kişi 2)	PostgreSQL bağlantısı (secret)
<code>REDIS_URL</code>	realtime	Redis bağlantısı (secret)
<code>SESSION_JWT_SECRET</code>	API (Kişi 2)	JWT imzalama anahtarı (secret)

Hata sözleşmesi

Tüm hatalar `{ code, message, retryable }` gövdesiyle döner. Kullanılan stabil kodlar (docs/ARCHITECTURE.md): `UNAUTHENTICATED` , `CONFLICT` , `PROFILE_INCOMPLETE` , `NOT_ELIGIBLE` , `RATE_LIMITED` . Tüm kod kümesi `server/src/types/api.ts` içindeki `ApiErrorCode` enum'unda tanımlıdır.

Her durum değiştiren uç nokta, isteğe bağlı bir `Idempotency-Key` (UUID) başlığı kabul eder; aynı anahtarla yapılan tekrar, orijinal sonucu döndürür.

Endpoint'ler

POST /v1/auth/register

Yeni hesap oluşturur.

İstek:

```
{ "email": "user@example.com", "password": "supersecret1" }
```

Yanıt 201 :

```
{ "userId": "uuid", "email": "user@example.com", "profileIncomplete": true, "token": "<jwt>" }
```

Hatalar: CONFLICT (409, e-posta zaten var), doğrulama hatası (400).

POST /v1/auth/login

Kimlik doğrular, JWT döner.

İstek: { "email": "...", "password": "..." }

Yanıt 200 : { "userId", "email", "profileIncomplete", "token" }

Hata: UNAUTHENTICATED (401, e-posta/parola hatalı — kullanıcı sızıntısı olmaması için aynı mesaj).

PUT /v1/profile (kimlik doğrulama gerekli)

display_name , birth_date , bio günceller.

İstek:

```
{ "displayName": "Ayşe", "birthDate": "1994-05-06", "bio": "merhaba" }
```

Yanıt 200 : profil alanları + profileIncomplete: false .

Hata: UNAUTHENTICATED (401).

PUT /v1/profile/preferences (kimlik doğrulama gerekli)

min_age , max_age günceller (maxAge >= minAge).

İstek: { "minAge": 20, "maxAge": 35 }

Yanıt 200 : { "userId", "minAge", "maxAge", "updatedAt" }

Hatalar: UNAUTHENTICATED (401), doğrulama hatası (400).

POST /v1/queue/join (kimlik doğrulama gerekli)

Eşleştirme havuzuna katılır. Idempotency-Key başlığı kabul eder.

Sunucu kontrolleri: kimlik doğrulanmış → profil tamamlanmış → uygunluk (yaş ≥ 18) → hız sınırı. Engellenmiş kullanıcılar her iki yönde de aday havuzundan çıkarılır (selectCandidate).

Yanıt 201 : { "userId", "status": "queued" | "matched", "joinedAt", "idempotentReplay": false }

Aynı Idempotency-Key tekrarı → 200 + idempotentReplay: true .

Hatalar: UNAUTHENTICATED (401), PROFILE_INCOMPLETE (409), NOT_ELIGIBLE (403, 18 yaş altı), RATE_LIMITED (429), CONFLICT (409, zaten kuyrukta).

POST /v1/queue/leave (kimlik doğrulama gerekli)

Kuyruktan çıkar. Yanıt 200 : { "userId", "status": "left" } .

Hata: CONFLICT (409, kuyrukta değil).

GET /health

{ "status": "ok" } döner (CI/uptime kontrolü).

Veritabanı tabloları (server/src/db/schema.sql)

Tablo	Alanlar
users	id , email (unique), password_hash , created_at
profiles	user_id (PK/FK), display_name , birth_date , bio , updated_at
preferences	user_id (PK/FK), min_age , max_age , updated_at
blocks	blocker_id , blocked_id , created_at (PK: çift)
queue_entries	user_id (PK/FK), joined_at , idempotency_key , status

status değerleri: queued / matched / left . Şema idempotent'tir (yeniden çalıştırılabilir). Parolalar Node `scrypt` ile hashlenir (harici bağımlılık yok).

Migration notu: Ayı bir migration aracı henüz eklenmedi; şema tek dosya olarak `psql "$DATABASE_URL" -f server/src/db/schema.sql` ile uygulanır. Tüm ifadeler `IF NOT EXISTS` kullandığından mevcut veriye zarar vermez.

Testler

```
cd server
npm test # vitest run - 21 test (auth + queue), gerçek DB gerekmez
npm run type-check
```

Testler bellek içi (in-memory) veri deposu (`createInMemoryStore`) ve enjekte edilen hız sınırlayıcı kullanır; PostgreSQL/Redis bağlantısı gerektirmez.

Alan sınırları (diğer kişilerle çakışma yok)

- **Kişi 2 (bu iş):** yalnızca `server/` klasörü, `docs/SPRINT1_KISI2.md` ve `.env.example` 'a backend değişkenleri. İstemci (`src/`) koduna dokunulmadı.
- **Kişi 1 (istemci):** `src/` , `index.html` , Vite yapılandırması. Aşağıdaki "İstemci entegrasyonu" örneğiyle bu endpoint'lere bağlanır.
- **Kişi 3 (WebRTC/CI):** `.github/workflows/ci.yml` , WebRTC spike, tehdit modeli. Backend testleri `cd server && npm test` komutuyla CI'a eklenebilir (aşağıya bakın).

Kişi 1 — istemci entegrasyon örneği

```
// Kayıt
const res = await fetch(`${import.meta.env.VITE_API_BASE_URL}/v1/auth/register`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password }),
});
const { token, profileIncomplete } = await res.json();

// Kuyruğa katılma (idempotent)
await fetch(`${import.meta.env.VITE_API_BASE_URL}/v1/queue/join`, {
  method: "POST",
  headers: {
    Authorization: `Bearer ${token}`,
    "Idempotency-Key": crypto.randomUUID(),
  },
});
```

Kişi 3 — CI'da çalıştırılacak test komutu

Mevcut `unit-and-integration-tests` işine backend adımı eklenebilir:

```
- name: Backend tests (Kişi 2)
  working-directory: server
  run: |
    npm ci || npm install
    npm run type-check
    npm test
```